TUTORIAL

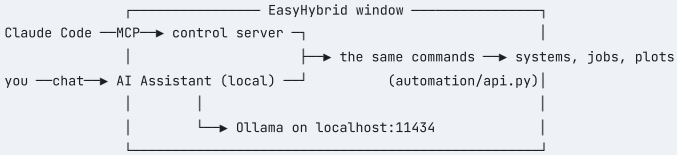
EasyHybrid with Claude Code and a local model (Ollama)

> Written for EasyHybrid 3.2.0 (2026-10-28). The answers and timings below come from real runs: Claude Code (Claude Opus/Sonnet), and Ollama with `gpt-oss:20b` on an RTX 3060 (12 GB).

EasyHybrid can be operated in plain language by two kinds of AI, at the same time and on the same session:

	Claude Code	**Local assistant (Ollama)**
Where it runs	Anthropic's servers (needs an account and internet)	your computer (Ollama, LM Studio, llama.cpp...)
How it reaches EasyHybrid	the MCP server, `src/automation/mcp_server.py`	**Edit → AI Assistant (Local Model)...** inside EasyHybrid
Good at	planning and running whole studies; choosing protocols; diagnosing what went wrong; writing reports	quick questions about the session and the logs; short routine tasks
Data	the tool results (numbers, file names, atom lists) go to Anthropic	nothing leaves the computer
Cost	tokens of your plan (reduced by the token options of EasyHybrid)	none (electricity)

Both use the **same commands** (`state` , `atoms` , `optimize` , `scan` , `umbrella` , `us_check` , `pes` , `job_summary` , remote jobs...) through the control layer described in AI control. They are not two separate programs to learn.



---

1. Set up EasyHybrid for AI control

1. Start EasyHybrid and tick **Edit → AI Control Server**. A dialog opens (Figure 1): - **Token use → Economical (recommended)**: short replies, results in the same call, small screenshots. This makes Claude Code several times cheaper and also suits the local model. - **Model that operates EasyHybrid**: leave *Same as the Claude Code session* for now (section 5 explains the other choices). - Press **Start**. The status bar shows the socket, `~/.easyhybrid/control.sock` .
2. Alternatively, start it from a terminal with `python3 easyhybrid.py --control` .

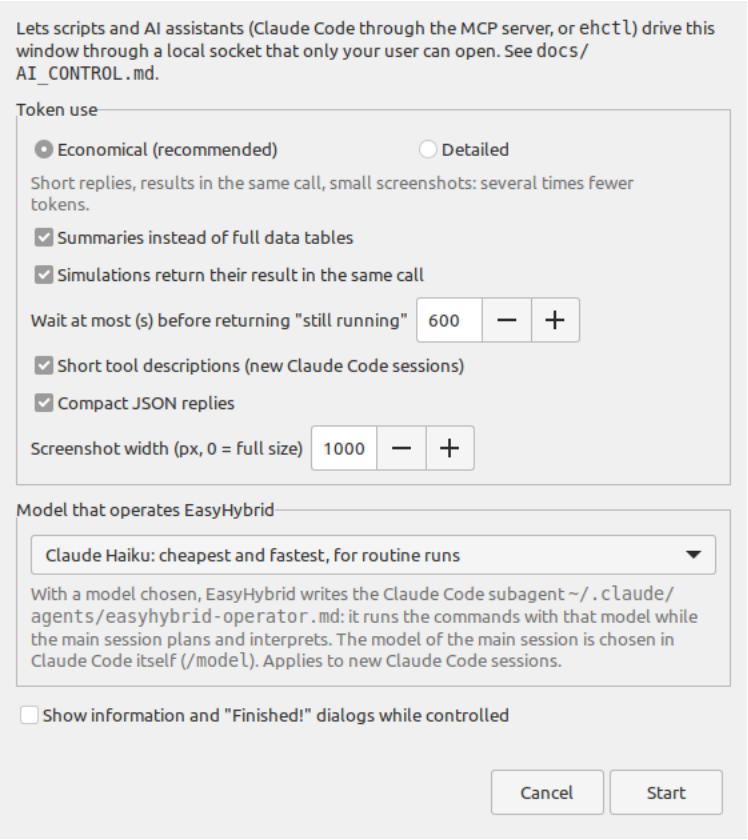


Figure 1. The options of the control server.

The socket can only be opened by your own user. Nothing listens on the network.

2. Connect Claude Code

In a terminal, once (replace the path by your EasyHybrid folder):

```
claude mcp add easyhybrid -- python3 /path/to/EasyHybrid/src/automation/mcp_server.py
claude mcp list          # easyhybrid: ... - ✔ Connected
```

Then open a **new** Claude Code session: the EasyHybrid tools appear as `mcp__easyhybrid__state`, `mcp__easyhybrid__scan`, and so on (`/mcp` inside the session lists them). Claude Code asks your permission the first time it uses each tool; read-only tools (`state`, `atoms`, `pes`, `job_summary`, `us_check`) can be allowed for good.

If EasyHybrid is not running, Claude can start it with the `launch` tool.

3. Install a local model (Ollama)

```
curl -fsSL https://ollama.com/install.sh | sh     # Linux; installers for macOS/Windows at ollama.com
ollama pull gpt-oss:20b                           # the model used here, 14 GB
ollama list
curl -s http://localhost:11434/api/tags | head -c 300    # the server answers
```

The model must support **tool calling**: EasyHybrid lists the models of the server and marks those with "tools". A model larger than the GPU memory still works, with part of it on the CPU, but more slowly. With `gpt-oss:20b` (14 GB) on a 12 GB card, the first answer, which also loads the model, took about 1.5 min, and the later ones 10–40 s.

Open **Edit → AI Assistant (Local Model)...** (Figure 2):

- **Ollama**, `http://localhost:11434`, the model chosen from the list (↻ refreshes it). An *OpenAI-compatible* server (LM Studio, llama.cpp, vLLM) works the same way.
- **Run actions without asking** off (the default). Reading is always free. Loading files, changing the QC region or starting a simulation shows a Yes/No dialog first.
- `python`, scripts, the setup of remote machines and screenshots are never offered to a local model.

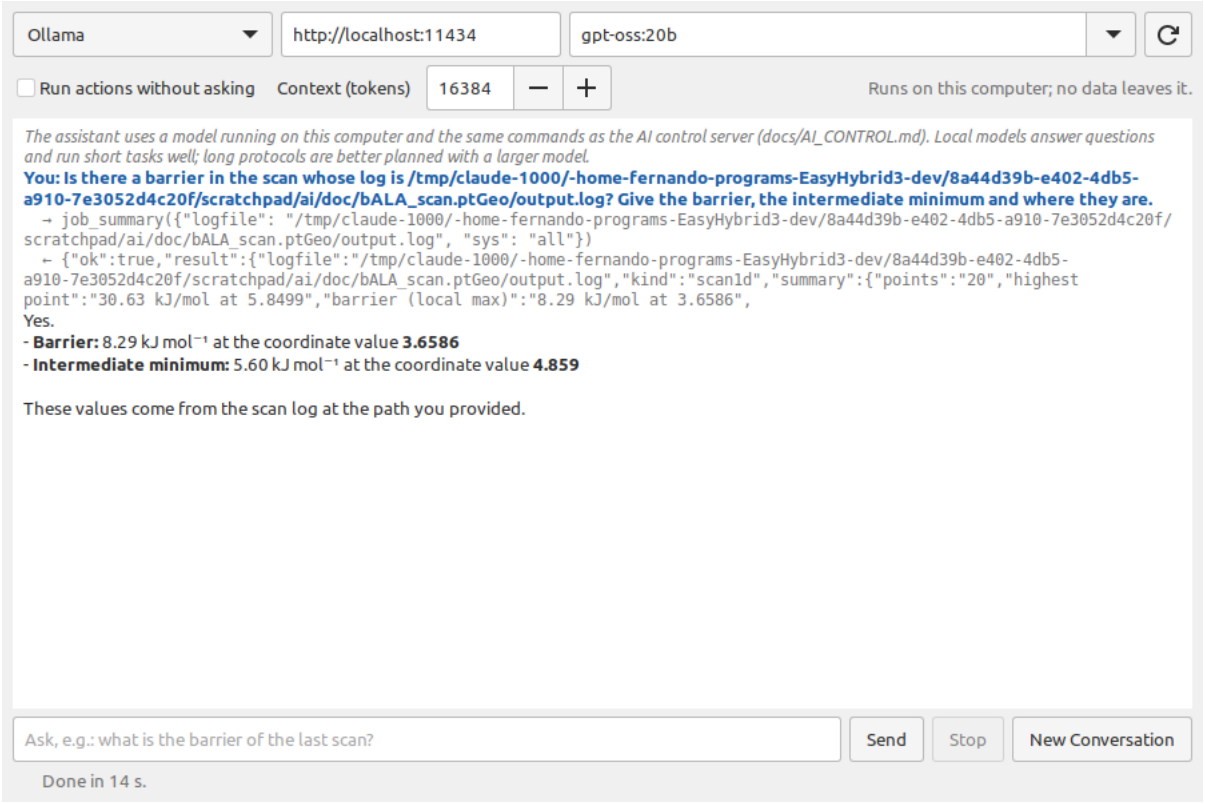


Figure 2. A question about a scan log, answered by `gpt-oss:20b` *in 14 s. In grey, the tool the model called and what it read.*

---

4. A study with both: the bALA hydrogen bond

This example follows the bALA tutorials, in plain language. It uses Claude Code for the study and the local assistant for the questions along the way.

4.1 Claude Code runs the study

In Claude Code, with EasyHybrid open:

> Load `examples/tutorials/01_bala/bALA_not_opt.mol` with OPLS in EasyHybrid, optimize it, and scan the O5–H17 distance from the C7eq hydrogen bond outwards in 20 steps of 0.2 Å. Tell me the barrier and the C5 minimum. Then run umbrella sampling from the scan with k = 100 on 8 CPUs, check it and give me the free-energy barrier.

Claude chains the commands below, and each one appears in the EasyHybrid window as it runs. The results are those of runs of the same commands on this system: the potential-energy steps with `ehctl`, and the umbrella sampling in the long run of the bALA umbrella sampling tutorial (20 000 + 50 000 steps per window):

Step	Command	Result
system	`new_system` (OPLS)	system 0, 22 atoms
optimization	`optimize`	converged, −116.2 kJ/mol
the atoms	`atoms name=O5,H17`, then `measure`	indices 5 and 17, 2.06 Å apart (the C7eq hydrogen bond)
scan	`scan` (distance 5–17, 0.2 Å × 20)	barrier **8.29 kJ/mol at 3.66 Å**, C5 minimum 5.60 kJ/mol at 4.86 Å
umbrella sampling	`umbrella` (from the scan, k = 100)	20 windows
diagnostics	`us_check`	**OK**: no window more than 3 σ from its target, lowest overlap 0.36
free energy	`wham`	barrier **5.7 kJ/mol**, C5 minimum 3.1 kJ/mol: about 2.5 kJ/mol below the potential energies

With *Economical* on, each simulation returns already imported and summarized, so the whole study takes a handful of calls. If a check fails (`us_check` says **fail**), Claude explains why and proposes the fix (more windows, another force constant) before going on.

4.2 The local assistant answers along the way

While the jobs run, or later, ask the local assistant inside EasyHybrid. It is free and private, and it reads the same session:

> What is the barrier of the last scan?
>
> Is the umbrella sampling done? Did it pass the checks?
>
> Summarize job 2 of system 0.

It calls `pes`, `job_summary`, `jobs` or `us_check` and gives the numbers (Figure 2). For a small action, for example "show the scan trajectory at frame 8" or "import the result of job 1", it asks you first.

4.3 The same numbers without any AI

A double click on a job in the Process Manager opens its log with the **Plots** tab (Figure 3): the profile or surface, the barrier, the convergence of each step, and the ORCA/xTB energies when a QC program was used. `job_summary` gives the AI the same summary.

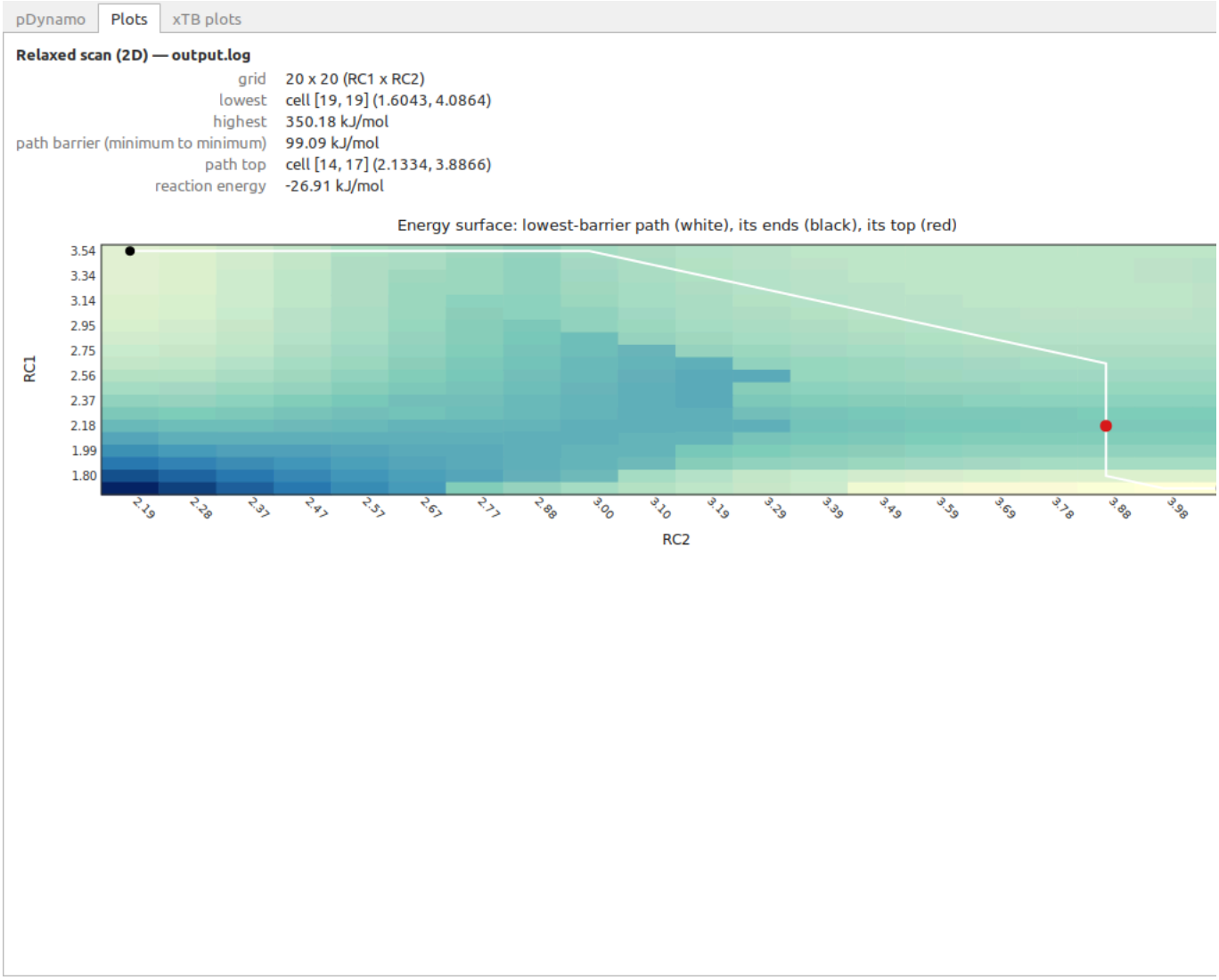


Figure 3. The Plots tab of a 2D xTB scan: the energy map with the lowest-barrier path, its ends and its top.

5. Who does what: the cheapest combination

Task	Best choice	Why
"What is the barrier / the temperature / the status of job 3?"	local assistant	instant, free, private
"Show residue 45 in sticks", "go to frame 12"	local assistant (or the terminal)	one command
Set up a QC/MM system, choose the QC region and method	Claude Code	needs chemical judgement
Plan a scan, NEB or umbrella sampling protocol (spacing, force constant, images)	Claude Code	many decisions; errors cost hours of computing
A run failed or does not converge: why?	Claude Code	reads logs, compares runs, proposes fixes

Task	Best choice	Why
Repeat a protocol you already validated on another system	Claude Code with a cheap operator (below), or the local assistant	routine
Write the methods or results of a study	Claude Code	text with the real numbers

Inside Claude Code you can lower the cost further: **Edit → AI Control Settings... → Model that operates EasyHybrid → Claude Haiku**. EasyHybrid then writes the subagent `~/.claude/agents/easyhybrid-operator.md`. Claude Code's main model (chosen with `/model`) plans and interprets, and the cheap subagent clicks the buttons.

6. Remote machines

Both assistants can work with the clusters defined in **Simulate → Remote Machines...** (see remote jobs):

- Claude Code: *"run that umbrella sampling on hpc1 with 12 CPUs"* sends it with `remote="hpc1"`. Later, *"is it done? check it"* fetches the results automatically and runs `us_check` and `wham`.
- The local assistant can follow the jobs (`remote_jobs`, read-only) and fetch or cancel them (asks first). Setting up a new machine stays with you or with Claude Code (*Detect Settings*).

7. Privacy

- With **Claude Code**, what the tools return goes to Anthropic as part of the conversation: numbers, atom names and indices, file paths, log summaries (no coordinates unless a command lists them). The structures and trajectories themselves stay on your disk.
- With the **local assistant**, nothing leaves the computer. For confidential projects, use only the local assistant, and turn the AI Control Server off (**Edit → AI Control Server**).

8. When something does not work

Symptom	Fix
Claude Code has no EasyHybrid tools	open a new session after `claude mcp add`; `claude mcp list` must say *Connected*
"EasyHybrid is not running with the control server"	tick **Edit → AI Control Server**, or ask Claude to `launch` it
A reply says `"pending"`	a question dialog is open in EasyHybrid: answer it, or let the AI use `dialog_respond`
The local assistant cannot list models	start the server (`ollama serve`) and press ↻; check the address
The model never calls a tool	choose a model marked "tools"; very small models often fail at this
The first local answer takes minutes	the model is being loaded (larger than the GPU: partly on the CPU); later answers are faster
The local model wanders between tools	ask more precisely ("read the log at …"); for multi-step work use Claude Code

Notes

- The commands, their arguments and the token options are described in `docs/AI_CONTROL.md`. Any script can use them too (`src/automation/ehctl.py`, or the Python client).
- The local assistant is `src/automation/local_agent.py` (standard library only), and works from a script: `LocalAgent("ollama", "http://localhost:11434", "gpt-oss:20b", execute, confirm).ask(...)`.

EasyHybrid 3.2.0 · built 2026-10-29 from `dev` @ `3539da4` · https://github.com/ferbachega/EasyHybrid3